

Name: _____

Written

- [5] 1. From the course notes, list the properties of a good *watermark*.

Solution:

- Difficult to remove [1]
- Contains as much information as possible [1]
- Does not corrupt or degrade the visible content [1]
- Is difficult to detect [1]
- Is encrypted [1]
- Is robust (survive transformation and transmission) [1]

- [5] 2. Why is `printf("Key Rejected")` a potentially unwise way to tell the user a license key was rejected?

Solution: A cracker may be able to quickly find your key checking algorithm [1]. The message tends to be displayed immediately after the check [1]. The cracker need only break on `MessageBox` [1] and try a bad key [1], then trace to how execution got there [1].

Name: _____

3. Consider the following C code output by some decompiler for a program you are evaluating for security *on Linux amd64*:

```

1:
2:  int unk_func(int param1)
3:
4:  {
5:      int iVar1;
6:      ssize_t sVar2;
7:      int param1-local;
8:      char local3 [1022];
9:      ushort local2;
10:
11:     local2 = 0;
12:     memset(local3,0,0x3fe);
13:     sVar2 = recv(param1,&local2,2,0x100);
14:     if (sVar2 < 0) {
15:         iVar1 = 0;
16:     }
17:     else if (local2 < 0x401) {
18:         sVar2 = recv(param1,local3,(long)(int)(local2 - 2),0x100);
19:         if (sVar2 < 0) {
20:             iVar1 = 0;
21:         }
22:         else {
23:             iVar1 = process_something(local3);
24:         }
25:     }
26:     else {
27:         iVar1 = 0;
28:     }
29:     return iVar1;
30: }
31:

```

- [1] (a) Suggest another name for param1.

Solution: sockfd (any reasonable name that describes the variable's *purpose* is acceptable.)

- [1] (b) Suggest another name for local2.

Solution: len

- [1] (c) Suggest another name for local3.

Solution: buf

- [5] (d) Diagram the intended format of the message received by this function.

Solution:

len (2 bytes)

data (len - 2 bytes)

Where len is two bytes in little-endian order, giving the length in bytes of the entire packet, including the header. data may be any arbitrary data of length len - 2 bytes.

- [5] (e) Locate and describe at least one vulnerability in the function, and briefly outline how it might be exploited. You may assume this is a system with no mitigations (i.e., DEP/NX bit, stack guards, ASLR, etc.)

Solution: While the length of the entire buffer is checked against 1024, it is not checked to be greater than 2, meaning large enough to hold its header. Thus, if 0 or 1 is given as the length, the second call

to recv will get `local2 - 2`, which will have underflowed to a large number as the length. We can thus specify 1 as the length and then overflow the buffer `local3`, smashing the stack.

- [4] (f) Give two or more programming recommendations to patch the vulnerability.

Solution: First, you may consider using dynamic memory allocation since packets can vary in size, though, this does not capture or eliminate the flaw in itself. You may either check that the given length is greater than 2, meaning it contains its header, or you can store `local2 - 2` in a temporary variable and check its range, too. Also, `(process_something` would probably need the length passed in as a parameter.

Name: _____

4. Consider the following disassembly:

```

00000000000101120: f3 0f 1e fa  ENDBR64
00000000000101124: eb 01        JMP      00101127
00000000000101126: 0f 48 8d 35  CMOVS   ECX,dword ptr [RBP + 0xed235]
                        d2 0e 00
0000000000010112d: 00 bf 01 00  ADD     byte ptr [RDI + 0x1],BH
                        00 00
00000000000101133: 31 c0        XOR     EAX,EAX
00000000000101135: e9 16 ff ff  JMP     00101050
                        ff

```

- [1] (a) What approach does this tool use to produce its disassembly?
- A. **Linear Sweep** B. Recursive Traversal C. Not enough information
- [3] (b) Which instruction is the first incorrect instruction displayed in this listing?

Solution: The cmovs starting at 1153.

- [5] (c) Explain how you determined your answer to part (b).

Solution: The first instruction is a jump to 1154, which occurs in the middle of the incorrect instruction. I would instead expect to see an instruction starting at 1154 preceded by an orphaned byte at 1153 having the value 0x0f. Without a manual or actual disassembler, I cannot tell you what those instructions should be.

- [4] (d) Explain why the other disassembly approach would not fall for this obfuscation trick.

Solution: *Recursive Traversal* disassembles only those instructions it encounters as a potential branching point. Because the “instruction” at 1153 follows an unconditional jump, a recursive traversal disassembler would not attempt to disassemble it. Instead it would see the branch location of 1154 and resume disassembly there.

Name: _____

Practical

5. You should have a copy of the `practical_malware` executable *for Linux amd64*. It is a "malware" sample, but for simplicity's sake, it has only one rather innocuous command. However, it does employ some more sophisticated anti-RE techniques.

(a) It employs a time-based anti-debugging technique.

[10] i. Explain how its implementation of the technique works.

[10] ii. Patch the executable to defeat the technique. Submit your patched copy with your exam.

(b) It connects to a remote command-and-control server.

[4] i. What is the hostname and port of the server?

[8] ii. Describe the protocol, i.e., what messages can it send and receive, and in what sequence?

[8] iii. Explain why the first message it receives is so critical.

(c) It decodes a portion of itself during execution.

[5] i. What is the "key" used to protect and decode this portion?

[10] ii. Decode and disassemble the protected portion. Submit a copy of the disassembly with your exam.

[5] iii. What does the protected portion do?